National University of Lesotho

Department of Mathematics and Computer Science

CS4433 Software Engineering

Individual Final Project

Moderated Learning Management System

Maintenance

Change request CR1 and release v1.1

Student Name: MAKHOTSO RAMOCHELA

Student Number: 202321185

1. Introduction

After version 1.0 was released, the school sent this change request:

CR1: "A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list."

This document has the classification and impact analysis (Section 2), the implementation as release v1.1 with the updated design documents (Section 3), and the regression testing (Section 4). Table 1.1 shows where each deliverable is.

Table 1.1: The deliverables of Phase 8

Deliverable	Where it is	Remark
Classification of the maintenance type	Section 2.1	
Impact analysis: requirements, diagrams, classes, tables and test cases	Sections 2.3 to 2.5	Also the other documents (Section 2.6)
Release v1.1	Section 3 and the Git tag v1.1	Code on the branch feature/cr1-class-extension, merged into main
Updated design documents	Section 3.4	Software_Design_Document_v3 and Requirements_Engineering_Phase2_v3
New tests and regression test results	Section 4	

2. Classification and impact analysis

2.1 Classification

Maintenance is classified in four types (Sommerville, 2016): corrective (a fault is repaired), adaptive (the program is changed for a new environment), perfective (new or better functions are added for the users) and preventive (the program is improved to avoid future trouble). Table 2.1 shows the decision.

Table 2.1: Classification of CR1

Type	Is CR1 this?	Reason
Corrective	No	Nothing is wrong. Version 1.0 does what the SRS says: FR-10 asks for an extension for one student. No test failed and no defect is open.
Adaptive	No	The operating system, the libraries and the school computers are the same.
Perfective	**Yes**	The school asks for a new function (an extension for a whole class) and for a better screen (extensions on the student's list). The users asked for it, and the program works correctly without it.
Preventive	No	The change is not made to stop a future fault or to clean the code.

CR1 is therefore **perfective maintenance**. There is one more point: the SRS listed "an extension for a whole class at once" as Won't have in version 1.0. So CR1 changes a requirement that was fixed at Gate 1. As the SRS says, a later change must be written in the change log, and Section 3.4 does this. The change is small: it adds one operation and one column, and it does not change the data.

2.2 Possible solutions

Table 2.2: Three ways to make an extension for a class

Solution	How it works	Good and bad points
A. One extension for each student (chosen)	The service saves one extension row for each enrolled student, in one transaction, by the same rules as for one student.	Good: no change to the database, the deadline rule (latest extension of the student) and BR-04 are reused, and each student keeps who gave the extension and when. Bad: more rows (one for each student), and a student who enrolls later does not get it.
B. One row for the class	A new extension row with no student (student_id empty). The deadline rule looks for a student row first and then a class row.	Good: one row and it covers late enrolments. Bad: the schema and the deadline rule change, the rule "later than the earlier extension" becomes difficult, and the tables, the ERD and the data dictionary change.
C. The menu repeats the single-student operation	The menu calls grantExtension for each student in a loop.	Good: very little code. Bad: it breaks architecture rule AR-5 (one change is one transaction): half of the class could get the extension if there is an error. It also puts the rule in the menu.

I chose **A**. It has the smallest effect on the design, it keeps all the old tests valid, and it keeps the rules of BR-04 in the service. The bad point (late enrolments) is accepted, written in the release notes, and solved by the existing single-student extension.

2.3 Affected requirements and use cases

Table 2.3: Impact on the requirements

Item	Impact	What changes, or why it does not
FR-10	Changed	"Give one student an extension" becomes "give one student, or all the students of a course, an extension". Priority stays Must.
FR-15	Changed	The list of assignments also shows the original deadline when the student's deadline was extended.
BR-04	Changed (note)	A note says that a class extension applies the rules of BR-04 to each student, and that a student who does not meet them is skipped. The rules themselves are the same.
UC-07	Changed	"Gives one student a later deadline" becomes "gives one student, or the whole class, a later deadline".
Won't have list	Changed	"An extension for a whole class at once" is removed from the list of version 1.0 and the list now says "versions 1.0 and 1.1".
UC-15, FR-13, BR-01	Checked, not changed	UC-15 and FR-13 use the deadline of the student. The deadline rule is not changed, so the penalty is the same. The wording of UC-15 already covers deadlines.
NFR-04, NFR-08, NFR-09, NFR-12	Checked, not changed	The new function uses the same right (GRANT_EXTENSION), so Table 10 does not change (NFR-04). Wrong input is handled as before (NFR-08). Audit entries are only for final marks (NFR-09). The new screens are text only (NFR-12).
NFR-06	Checked, not changed	The four tasks that must take 7 steps or less (submit, mark, approve, release) are not touched.

2.4 Affected diagrams, classes and tables

Table 2.4: Impact on the design

Item	Impact	What changes, or why it does not
Figure 3.5 (AssignmentService, AuditService, MessageService)	Changed	AssignmentService has the new operation grantClassExtension. The figure also shows the two dependencies (users and penalty) that the earlier figure missed.
Figure 4.8	New	Sequence diagram: give an extension to the whole class.

Item	Impact	What changes, or why it does not
Figure 3.2 (assignments and extensions), Figure 3.3	Checked, not changed	The class Extension is the same: one extension is for one student.
Figure 4.1 (state machine)	Checked, not changed	An extension for a student with no submission or a Submitted one changes only the deadline. For a Late submission the existing transition Late to Submitted is used (BR-04).
Figures 4.3 to 4.5 (submit)	Checked, not changed	They use deadlineFor, which is not changed.
Figure 5.1 (ERD), data dictionary	Checked, not changed	No table or column changes. The table extension gets more rows.
Component and deployment diagrams, use case diagram	Checked, not changed	No new component or device. UC-07 keeps its name.
Class AssignmentService	Changed	New public operation grantClassExtension. Two new private operations: checkExtension (the rules of BR-04 for one student) and applyExtension (saves one extension and works out late work again). grantExtension uses them.
Class SubmissionService	Changed (small)	listMyAssignments fills two new fields.
Type StudentAssignmentRow	Changed	Two new fields: originalDeadline and extended.
Type ClassExtensionResult	New	granted, skippedNotLater, skippedMarked.
Classes LecturerMenu and StudentMenu	Changed	LecturerMenu has choice 8. StudentMenu shows the new column.
Extension, IAssignmentRepository, ICourseRepository, SqliteAssignmentRepository, PenaltyStrategy, UnitOfWork	Checked, not changed	The existing operations are enough: addExtension, latestExtension, listStudentIds, begin and commit.
Table extension (and its index)	Not changed	The index on (assignment_id, student_id) already serves the deadline lookup. A class of 100 students adds 100 rows.
Table 3.2 (supporting types), traceability table	Changed	New type, new fields, and the rows FR-10 and FR-15.

2.5 Affected test cases

Table 2.5: Impact on the tests

Test cases	Impact	Reason
Unit and integration: extension_rules, extension_makes_late_work_on_time_again, extension_recalculates_the_days_late_when_still_late	At risk, re-run	grantExtension was changed inside (it now uses checkExtension and applyExtension), so these are the first regression checks.
Unit and integration: student_assignment_list_shows_the_deadline_of_the_student_and_not_submitted	At risk, re-run	listMyAssignments was changed.
System: ST-34 to ST-37 (extension for one student)	At risk, re-run	The single-student extension was changed inside.
System: ST-38 to ST-40 (student list)	At risk, re-run	The list has a new column, so the screen text changed.
System: all tests that use the Lecturer menu	At risk, re-run	The menu has a new choice. It was added at the end so that the numbers 1 to 7 stay the same.
All other tests (36 unit and integration, 95 system, scripted session, wrong input)	Re-run	A change in a shared service needs the whole suite to be run again.
New: 8 unit and integration tests and 8 system tests	New	They test the new function and the new column (Section 4.2).

2.6 Other documents and risks

The following were also affected: the user manual of the Lecturer (a new task), the user manual of the Student (the new column), README.md, docs/TRACEABILITY.md, the release notes and the version number (1.1.0 in CMakeLists.txt). The installation guide, the database schema and the backup steps are not affected.

Table 2.6: Risks of the change

Risk	How it is handled
The single-student extension stops working because the code was changed inside	The old tests for it are not changed and are re-run. The new code shares checkExtension and applyExtension, so there is one copy of the rules.
Half of the class gets the extension after an error	All the extensions are saved in one transaction. If nobody can get it, the service throws and the transaction is rolled back. A test checks that nothing is saved.
The numbers of the menu choices change and the manuals and tests become wrong	The new choice is number 8, after the old ones.
A lecturer gives a class extension by mistake	The lecturer must confirm, and must type a reason. Each extension is saved with who gave it and when. The program has no way to cancel an extension, so the lecturer can only give a later one. This is a known limit.
Students who enrol later do not get the extension	Accepted for version 1.1 and written in the release notes. The lecturer gives that student an individual extension.

3. Implementation: release v1.1

3.1 How the new function works

- **Lecturer menu, choice 8, "Give an extension to the whole class".** The lecturer types the assignment Id, the new deadline and a reason, and confirms with y.
- **The service checks** the role (GRANT_EXTENSION), that the lecturer teaches the course, and that the reason is not empty.
- **For each student enrolled in the course**, in one transaction, the service applies the rules of BR-04: the new deadline must be later than the student's present deadline, and the work must not be marked yet. A student who meets the rules gets an extension row. A student who does not is skipped and counted.
- **Late work is worked out again.** If a student's submission was Late and is now on time, it becomes Submitted again (BR-04).
- **If nobody can get the extension**, the service throws an error and nothing is saved.
- **The screen tells the lecturer** how many students got the extension, how many already had the same or a later deadline, and how many were skipped because the work was marked.
- **The student's list** has a new column "Original deadline". It shows the assignment's own deadline when the student's deadline was extended, and "-" when it was not.

3.2 Changes to the code

Table 3.1: Files changed

File	Change
src/business/AssignmentService.h, .cpp	New operation grantClassExtension. New private operations checkExtension and applyExtension. grantExtension uses them.
src/business/Dto.h	StudentAssignmentRow has originalDeadline and extended. New type ClassExtensionResult.

File	Change
src/business/SubmissionService.cpp	listMyAssignments fills the two new fields.
src/ui/LecturerMenu.h, .cpp	Menu choice 8 and the method grantClassExtension.
src/ui/StudentMenu.cpp	The new column in the list.
src/main.cpp, CMakeLists.txt	Version 1.1 (1.1.0 in CMake).

The new functions are short and simple: grantClassExtension has a complexity of 5, checkExtension 5, applyExtension 3, and the menu method 4. grantExtension went down from 10 to 6 because it shares the rules. No function is over the limit of 10. The layer rules are not changed: the menu calls only the service, and the service uses only the existing interfaces. No new operation was needed in the repositories.

3.3 Configuration management

Table 3.2: Git history of the change

Commit	Message	Branch
bde27ef	CR1: extension for the whole class and the original deadline on the student's list	feature/cr1-class-extension
7dfef38	CR1: make the system tests ST-99 and ST-101 specific, add the CI log of version 1.1	feature/cr1-class-extension
1a3d5a1	Merge feature/cr1-class-extension: change request CR1, version 1.1 (tag v1.1)	main

The work was done on the branch feature/cr1-class-extension and merged into main with git merge --no-ff, as in the branching strategy. The annotated tag **v1.1** is on the merge commit. A clean copy of the tag builds with no warnings and passes the unit and integration tests.

3.4 Updated design documents

Table 3.3 lists the documents that were updated. The old versions are kept, so that the changes can be compared.

Table 3.3: Updated documents

Document	What was changed
Software_Design_Document_v3 (was v2)	Title page and introduction: a note about the revision. Figure 3.5 redrawn with grantClassExtension. Table 3.2: new fields of StudentAssignmentRow and the new type ClassExtensionResult. New Figure 4.8 and its description. Table 5.7: a note that a class extension is saved as one row for each student. Traceability table: rows FR-10 and FR-15.
Requirements_Engineering_Phase2_v3 (was v2)	A change record in the introduction. FR-10 and FR-15. The note in BR-04. UC-07. The Won't have list. The label of FR-10 in the traceability table. This is the entry in the change log that the SRS asks for.
docs/TRACEABILITY.md, README.md, docs/RELEASE_NOTES.md	FR-10 and FR-15 rows, the new test names and numbers (44 and 103 tests), and the release notes of v1.1.
User manuals (Phase 7 document)	The new text is in Section 3.5.

Figures 3.1 and 3.2 show the new Figure 3.5 and the new Figure 4.8 as they are in the design document v3.

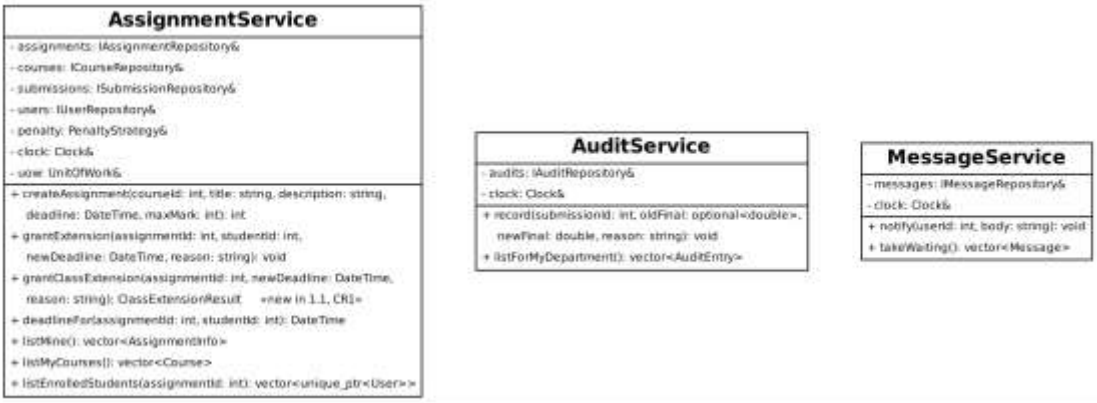


Figure 3.1: Figure 3.5 of the design document v3, with the new operation grantClassExtension

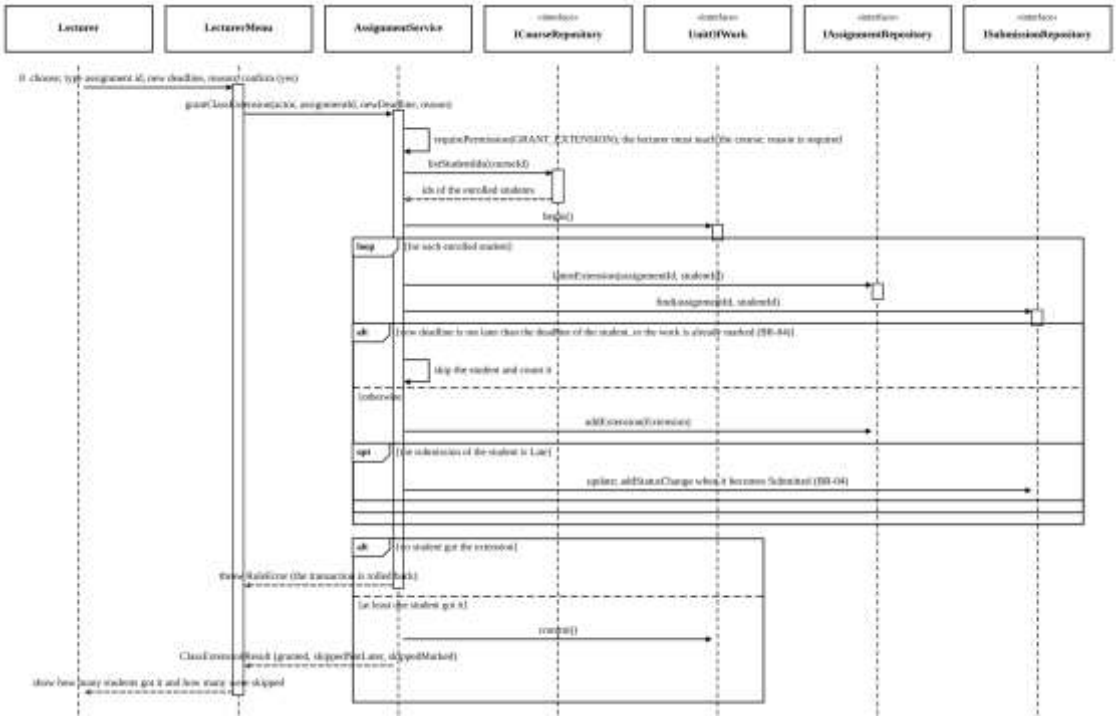


Figure 3.2: Figure 4.8 of the design document v3, give an extension to the whole class

3.5 Updated text of the user manuals

Lecturer, a new task: to give an extension to the whole class.

Step 1. Choose 8. The list of your assignments shows.

Step 2. Type the **Id** of the assignment, the **new deadline** (for example 2026-11-30 17:00) and the **reason**.

Step 3. Type **y** to confirm. The screen shows how many students got the extension and how many were skipped. A student is skipped if the student already has the same or a later deadline, or if the work is already marked.

A student who enrols after this does not get the extension. Give that student an extension with choice 4.

Student, choice 1: the list now has the column **Original deadline**. If your deadline was extended, your own deadline is shown under "Your deadline" and the first deadline of the assignment under "Original deadline". If there is no extension, the column shows a dash.

4. Regression testing

4.1 Plan

A change in a shared service can break functions that look unrelated. So the plan was: (1) keep all the old tests exactly as they were, and add no change to them; (2) add new tests for the new function and the new column; (3) run the whole suite again, together with the scripted session, the wrong-input test, the coverage check and the static analysis; (4) check that the new system tests fail on version 1.0, to show that they really test the change. The old test code was not edited: the only changes to existing test files are the new test file in the CMake list and new cases added at the end of the system test script.

4.2 New tests

Table 4.1: New unit and integration tests (tests/test_class_extension.cpp)

Test	Rule	What it checks
every_student_of_the_course_gets_the_new_deadline	CR1, FR-10	All 3 students get the deadline; the extension keeps who gave it and why.
the_assignment_deadline_itself_does_not_change	CR1	Only the students' deadlines change.
late_work_is_worked_out_again_for_each_late_student	BR-04	Late students become Submitted, and the third student can submit on time.
students_with_a_later_deadline_or_marked_work_are_skipped	BR-04	1 granted, 1 skipped (later deadline), 1 skipped (marked).
rules_of_role_course_reason_and_date	AR-4	Another lecturer, a student and a HoD are refused; an empty reason and an unknown assignment are refused.
nothing_is_saved_when_no_student_can_get_it	AR-5	An earlier date or the same date gives an error and no extension row.
a_second_class_extension_must_be_later_than_the_first	BR-04	The same date again is refused; a later one is accepted.
the_assignment_list_of_the_student_shows_the_extension	FR-15	The fields extended and originalDeadline are right after one and after a class extension.

Table 4.2: New system tests (tests/system/run_system_tests.py)

Test	Case	Expected
ST-96	Extension for the whole class	Saved for every enrolled student (the database holds one row for each).
ST-97	The student sees the extension on the list	The new deadline and the column Original deadline are shown.
ST-98	A date that is not later (exception)	"No student can get this extension." and no data saved.
ST-99	No reason (exception)	"The reason must not be empty." and no data saved.
ST-100	The lecturer does not confirm	"Nothing was saved."
ST-101	Another lecturer's assignment (exception)	"You do not teach the course of this assignment." and no data saved.
ST-102	Late work becomes on time again (BR-04)	The submission is Submitted again, with 0 days late.
ST-103	The Lecturer menu (regression)	Choices 1 to 7 are as before, and choice 8 is new.

4.3 Results

Table 4.3: Version 1.0 compared with version 1.1

Check	Version 1.0	Version 1.1
Unit and integration tests	36 of 36 pass	44 of 44 pass (36 old, 8 new)
System tests	95 of 95 pass	103 of 103 pass (95 old, 8 new)
Old tests that fail on version 1.1	-	**0**
Scripted session (14 checks)	Pass	Pass
Wrong-input test (60 sessions)	0 problems	0 problems
Coverage of the business layer	85.74% (824 of 961 lines)	86.20% (856 of 993 lines)
Coverage of AssignmentService.cpp	68.4%	76.2% (99 of 130 lines)
Static analysis (clang-tidy, clang-format, cppcheck, complexity)	No findings	No findings
Build warnings	0	0

All the steps of the CI pipeline were run on my computer with tests/run_ci_locally.sh on the commit bde27ef, and all steps passed (the log is docs/CI_LOCAL_LOG_V1_1.txt). The system tests were then run again after the last change to the test script (ST-99 and ST-101), and 103 of 103 passed. The figures for version 1.0 are from its local CI run (docs/CI_LOCAL_LOG.txt), which was made on version 1.0 before the tag.

4.4 Do the new tests test the change?

The new system tests were run against the program of version 1.0. **All 8 failed**, and the 95 old tests passed. So the new tests detect the missing function. In the first version of the tests, ST-99 and ST-101 also passed on version 1.0, because they only looked for the word "Error", and version 1.0 shows an error for a choice that does not exist. That was a weak test. I changed both tests to look for the exact message ("The reason must not be empty." and "You do not teach the course of this assignment."), and they now fail on version 1.0 and pass on version 1.1.

4.5 Conclusion of the regression test

Version 1.1 adds the new function without breaking any earlier function: every old test passes without being changed, the coverage is higher, and no new finding appears in the static analysis. No defect was found in version 1.1 during these tests.

References

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.